KINFB40U: Architecture – Travaux Pratiques

2022-2025

1 TP1 : prise en main et programmes simples

1.1 Exercice 1

Reprendre le code, vu en TD, réalisant la fonction max de 2 registres à l'aide d'instructions conditionnelles. Compiler et charger ce code. Modifier, **dans le simulateur**, les valeurs dans les registres r1 et r2, puis exécuter le code en mode pas à pas. Tester avec plusieurs valeurs.

1.2 Exercice 2

Écrire un programme qui calcule le PGCD de 2 nombres contenus dans les registres r5 et r6 (les registres peuvent être initialisés directement dans le debugger, ou au début du programme). On supposera que la valeur dans r5 est supérieure ou égale à celle dans r6.

Rappel de l'algorithme (on suppose A $\geq$ B):

```
while (B != 0) {
        R = A;
        while (R ≥ B)
                R = R - B;
        A = B
        B = R
}
PGCD = A;
```

Exemples :
PGCD(45,25) = 5 (en hexa : PGCD(0x2d,0x19) = 0x5)
PGCD(48,18) = 6 (en hexa : PGCD(0x30,0x12) = 0x6)
PGCD(24,8) = 8 (en hexa : PGCD(0x18,0x8) = 0x8)

1.3 Exercice 3

Reprendre le programme réalisant les sommes des multiples de 2, 4 et 8. Utiliser la solution avec un décalage à droite (le bit de poids faible va dans le bit C de

CPSR). Vérifier à l'exécution la valeur de CPSR lorsque le nombre traité est un multiple.

```
MOVS R1, R1, LSR #1 @ met le bit 0 de R1 dans le bit C de CPSR et décale
                    @ R1 vers la droite
```

2 TP2 : accès mémoire

2.1 Exercice 4

Soit la définition suivante :

```
operateur: .byte '+' @ peut aussi contenir '*' ou '-'
.align
```

Ecrire un programme qui lit l'opérateur en mémoire et calcule, dans R3, l'application de l'opérateur à R1 et R2.

Par exemple, si la case operateur de la mémoire contient le caractère '-', le programme doit mettre dans R3, le résultat de la soustraction de R1 et R2. A vous d'initialiser les 3 paramètres (operateur, R1 et R2) pour tester votre programme.

2.2 Exercice 5

Ecrire un programme qui recherche le maximum dans un tableau de 10 entiers signés. Utiliser le mode d'adressage par pointeur. Le résultat est le numéro de l'élément le plus grand (le premier si plusieurs éléments ont la valeur du maximum).

Appliquer au tableau suivant :

```
tab: .int 9, -4, 27592, 0, -27592, 9, -4, 27592, 0, -27592
```

2.3 Exercice 6

Ecrire un programme qui transforme tous les caractères d'une chaîne en majuscules (la chaîne initiale peut déjà contenir quelques majuscules).

Rappel : une chaîne de caractères est un tableau d'octets, dont chacun est le code ASCII d'un des caractères. La fin de la chaîne est signalée par le code ASCII 0 (à ne pas confondre avec le code ASCII du caractère 0, noté '0' et égal à 0x30).

Indications :

- un caractère est une lettre minuscule si son code ASCII est compris entre 'a' ('a' représente le code ASCII de la lettre a) et 'z'.
- 'a'-'A' = 'b'-'B' = 'c'-'C' ... donc on peut passer du code ASCII d'une lettre minuscule à celui de la lettre majuscule correspondante en lui soustrayant 'a'-'A'.

Vous comprendrez mieux en regardant la Table des caractères ASCII (via une recherche sur google).

Test :

Vous testerez votre programme sur la chaine suivante :

chaine: .asciz "#ABZ]aez}"

Après exécution de votre programme, la chaine devra contenir :

#ABZ]AEZ}

2.4 Exercice 7

La mémoire étant linéaire, une matrice est rangée une ligne derrière l'autre dans la mémoire.

On considère une matrice de dimension m x n rangée en mémoire à l'adresse MAT. En supposant que r0 et r1 contiennent les numéros de ligne et de colonne i et j, **écrire le code (limité à 5 instructions hors initialisation des registres r0 et r1) permettant d'initialiser à 0 l'élément MAT[i,j]** :

- si les éléments sont des octets : écrire 0 dans MAT[1][2] avec la matrice suivante
  MAT : .byte 0, 1, 2, 3
  .byte 4, 5, 6, 7
- si les éléments sont des entiers (en ne modifiant qu'une instruction) : écrire 0 dans MAT[1][2] avec la matrice suivante
  MAT : .int 0, 1, 2, 3
  .int 4, 5, 6, 7

2.5 Exercice 8

Ecrire un programme qui recopie le contenu d'une zone mémoire dans une autre. Les adresses respectives des zones sont dans r1 (zone de départ) et r2 (zone d'arrivée). La taille d'une zone est de N octets. Envisager tous les cas possibles

: les zones se chevauchent ou pas, la zone de départ est située plus bas ou plus haut que la zone d'arrivée dans la mémoire.

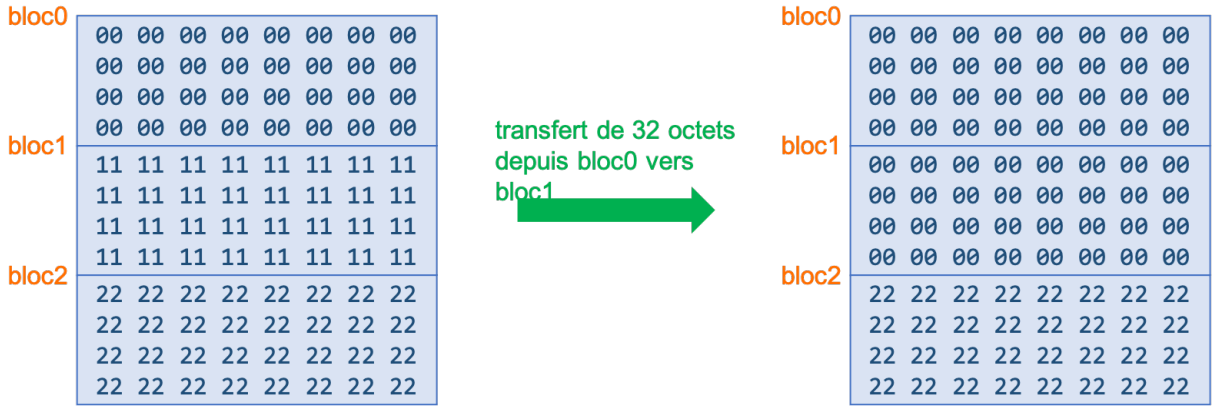
Dans un premier temps, faire des transferts octet par octet, puis optimiser le code en transférant des mots, des demi-mots et des octets de manière à minimiser le nombre de transferts.

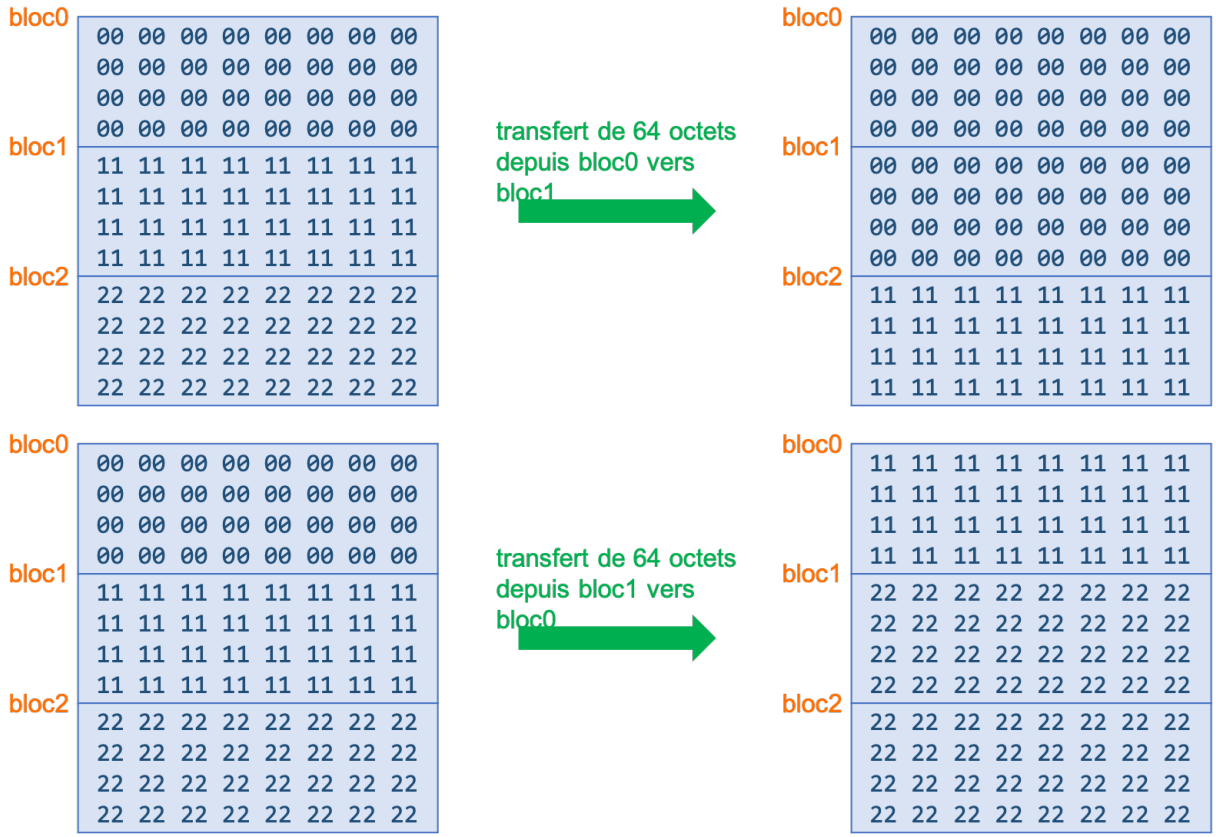
Pour tester votre programme, vous pouvez réserver trois blocs consécutifs de mémoire de taille 32 octets, initialisés respectivement à 0x00, 0x11 et 0x22 :

```
bloc0: .fill 32,1,0x00
bloc1: .fill 32,1,0x11
bloc2: .fill 32,1,0x22
```

Vous pouvez alors tester les transferts suivants :

- 32 octets depuis l'adresse bloc0 vers l'adresse bloc1
- 64 octets depuis l'adresse bloc0 vers l'adresse bloc1
- 64 octets depuis l'adresse bloc1 vers l'adresse bloc0
- 31 octets depuis l'adresse bloc0 vers l'adresse bloc1





3 TP3 : Sous-programmes

3.1 Exercice 9

- Ecrire un sous-programme ABS qui reçoit en entrée (dans le registre r1) un entier signé et renvoie (dans r0) sa valeur absolue. On utilisera l'instruction RSB et pas l'instruction MUL. Ce sous-programme peut ne faire que 4 instructions...
- Ecrire un programme qui calcule la somme des valeurs absolues des 10 éléments (entiers signés) d'un tableau. La valeur absolue d'un élément sera calculée en appelant le sous-programme ABS.

Appliquer au tableau suivant :

tab: .int 9, -4, 27592, 0, -27592, 9, -4, 27592, 0, -27592

Le résultat doit être : 110394 (0x1AF3A).

3.2 Exercice 10

On veut écrire un programme qui reçoit en entrée une date (sous la forme jour - mois - année) et calcule le quantième (c'est-à-dire le numéro du jour dans l'année, compris entre 1 et 366) correspondant.

Exemples : le quantième du 3 mars 2001 est 62. Le quantième du 3 mars 1996 est 63. Le quantième du 8 janvier 2004 est 8.

Dans un premier temps, on ne tient pas compte du fait que certaines années sont bissextiles.

- Ecrire le sous-programme DEBUT qui reçoit dans le registre r5 un numéro de mois (valeur entière) et renvoie dans r0 le quantième du jour précédant ce mois. Les valeurs de ces quantièmes sont indiqués ci-dessous. On suppose que le numéro de mois passé en paramètre est toujours correct, c'est-à-dire compris entre 1 et 12.

- Ecrire le programme principal qui calcule (en appelant le sous-programme DEBUT) le quantième d'une date dont les numéros de jour, de mois et d'année sont rangés en mémoire (3 variables). Le résultat sera rangé en mémoire dans une 4ème variable.

Mois	1	2	3	4	5	6	7	8	9	10	11	12
Quantième	0	31	59	90	120	151	181	212	243	273	304	334

Vous aurez intérêt à utiliser le tableau suivant :

quant: .int -1, 0, 31, 59, 90, 120, 151, 181, 212, 243,273,304,334

On veut maintenant modifier ce programme pour gérer correctement les années bissextiles.

- Ecrire le sous-programme DIVISIBLE qui reçoit deux valeurs entières a et b passées dans r5 et r6 et renvoie dans r0 le résultat 1 si a est multiple de b et 0 sinon.

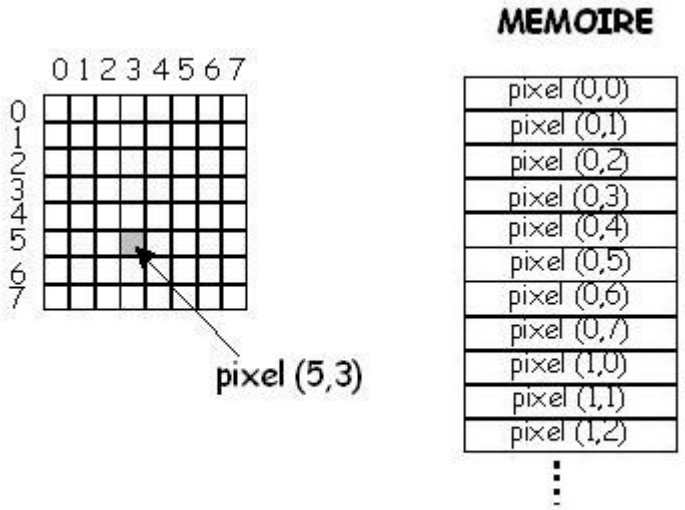
- Ecrire le sous-programme BISSEXTILE qui reçoit dans r5 un numéro d'année (valeur entière) et renvoie dans r0 le résultat 1 si l'année est bissextile et 0 sinon. Une année est bissextile si elle est divisible par 4 mais pas par 100 ou si elle est divisble par 400. Ce sous-programme devra appeler le sous-programme DIVISIBLE.

- Ecrire le programme principal.

3.3 Exercice 11

On veut traiter des images de 16 x 8 pixels. Chaque pixel est codé sur un octet qui peut prendre une des deux valeurs suivantes : 0 ou 35 (caractère '').

Une image est rangée ligne par ligne en mémoire (NB : ce schéma montre une image 8x8) :



Pour tester les programmes, on pourra utiliser l'exemple suivant :

```
. . . . . . # # # # . . . . . .
. . . . . . # # # # . . . . . .
. . . . # # # # # # # # . . . . .
. . . . # # # . . # # # . . . . .
. . . . # # # . . # # # . . . . .
. . . . # # # # # # # # . . . . .
. . . . . . # # # # . . . . . .
. . . . . . # # # # . . . . . .
```

dont la déclaration en assembleur est disponible sur la page moodle du cours. Lorsque l'on calcule l'érosion d'une image, tout pixel (hors bordure) qui possède au moins un voisin blanc (code ASCII 0) devient blanc. Les voisins sont les 8 pixels autour, ce qui inclut les pixels en diagonale. Les bordures de l'image ne sont pas modifiées.

- Ecrire un sous-programme qui calcule le résultat de l'érosion sur un pixel. Ce sous-programme aura les paramètres suivants :
  - en entrée : adresse du pixel dans l'image d'origine (passée dans un registre)
  - en sortie : valeur du pixel après érosion (dans r0)

- Ecrire un programme qui calcule l'érosion d'une image.

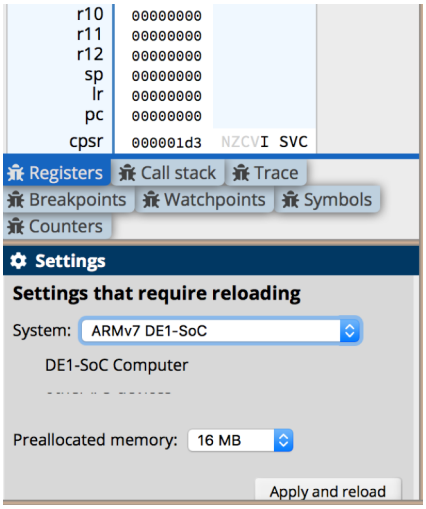
Dans le cas de l'exemple proposé ci-dessus, le résultat doit être :

```
. . . . . . # # # # . . . . . .
. . . . . . . # # . . . . . . .
. . . . . . . . . . . . . . . .
. . . . . # . . . . # . . . . .
. . . . . # . . . . # . . . . .
. . . . . . . . . . . . . . . .
. . . . . . . # # . . . . . . .
. . . . . . # # # # . . . . . .
```

4 TP4 : Programmation des entrées/sorties

Les systèmes informatiques sont généralement constitués, en plus du processeur et de la mémoire, de dispositifs périphériques permettant d'interagir avec l'utilisateur.

Sur le simulateur, on peut choisir une carte cible, la carte ARMv7 DE1-SoC (sélectionner la carte dans le menu déroulant puis cliquer sur le bouton "Apply and Reload" qui se trouve en dessous) :



Cette carte comporte un certain nombre de périphériques, parmi lesquels :

- des LEDs
- des boutons poussoir (push buttons)
- des afficheurs 7-segments

On peut contrôler ces dispositifs par programme, en lisant/modifiant des registres d'entrée/sortie (E/S) qui se trouvent dans leur interface.

Ces registres d'E/S ne sont pas connus du processeur, et ne peuvent donc pas être désignés directement (par leur nom) dans un programme. Par contre, ils sont associés à une adresse qui n'est pas utilisée par la mémoire (si les adresses sont sur 32 bits, elles peuvent désigner 4 GO - si on a moins de 4 GO de mémoire, on peut utiliser certaines adresses pour désigner autre chose q'un octet en mémoire). L'accès à ces registres d'E/S se fait donc par des instructions ldr/str aux adresses qui leur correspondent.

4.1 Contrôle des LEDs

Un registre associé à l'adresse 0xff200000 contient l'état des LEDs. Quand le bit i de ce registre est à 1, la LED correspondante est allumée ; quand il est à 0, la LED est éteinte.

Pour allumer la LED 0, on peut donc écrire :

```
ldr r8,=0xff200000 @ r8 = 0xff200000
                   @ on ne peut pas écrire mov r8,#0xff200000
mov r0,#1 @ un 1 en position 0 pour allumer la LED 0
str r0,[r8] @ on écrit les valeurs souhaitées pour les LEDs dans le registre
            @ de contrôle des LEDs
```

On obtient alors le résultat suivant :

Autre exemple : pour allumer les LEDs 1 et 2, on doit écrire un 1 en position 1 et en position 2, soit la valeur 0b110 (ou 0b0000000110) dans le registre de contrôle des LEDs.

4.2 Contrôle des boutons poussoirs

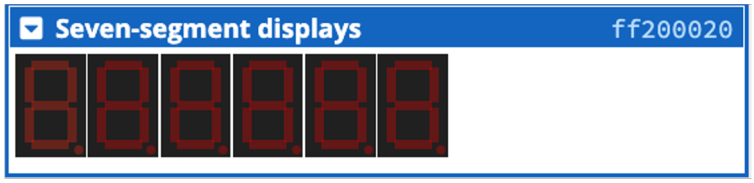
Pour les 4 boutons poussoir, un registre similaire se trouve à l'adresse 0xff20005c. Le bit i de ce registre indique si le bouton i a été pressé (dans ce cas, le bit est à 1). Par exemple, si on a pressé les boutons 1 et 3, le registre contient 0b0000...0001010.

Dans le simulateur, pour presser un bouton, il faut cliquer deux fois : la première fois, le carré devient bleu ; la deuxième fois, il redevient blanc. Le bit correspondant passe alors à 1 et y reste tant que le programme ne le remet pas à 0.

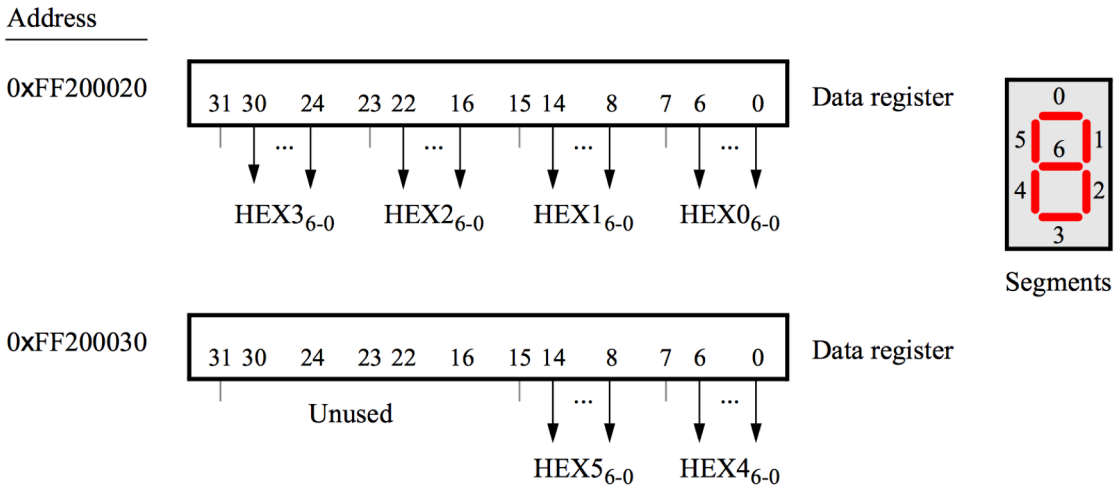
Pour mettre le bit i à 0, il faut écrire un 1 en position i dans le registre.

4.3 Contrôle des afficheurs 7-segments

Il y en a 6 et ils sont contrôlés par deux registres, l'un à l'adresse 0xff200020, qui contrôle les afficheurs 0 à 3, et l'autre à l'adresse 0xff200030, qui contrôle les afficheurs 4 et 5.



Pour chaque afficheur, il y a 7 bits à positionner comme représenté sur le schéma suivant :



Par exemple, pour afficher 0 sur l'afficheur 0, c'est-à-dire allumer les segments 0 à 5 de cet afficheur, on peut écrire :

```
ldr r8,=0xff200020
```

```
mov r0,#0b0111111 @ segment 6 éteint, segments 5 à 0 allumés
str r0,[r8]
```

Valeurs à écrire pour afficher les chiffres de 0 à 3 :

Chiffre	0	1	2	3
Valeur	0b0111111	0b0000110	0b1011011	0b1001111

4.4 Travail à réaliser

4.4.1 Fonctions d'attente

- Ecrire un sous-programme wait1 qui permet d'attendre un laps de temps de l'ordre de une seconde. Ce sous-programme devra simplement compter de 0 à une valeur prédéfinie (ou décompter de cette valeur jusqu'à 0). Ce sous-programme n'a pas de paramètre.

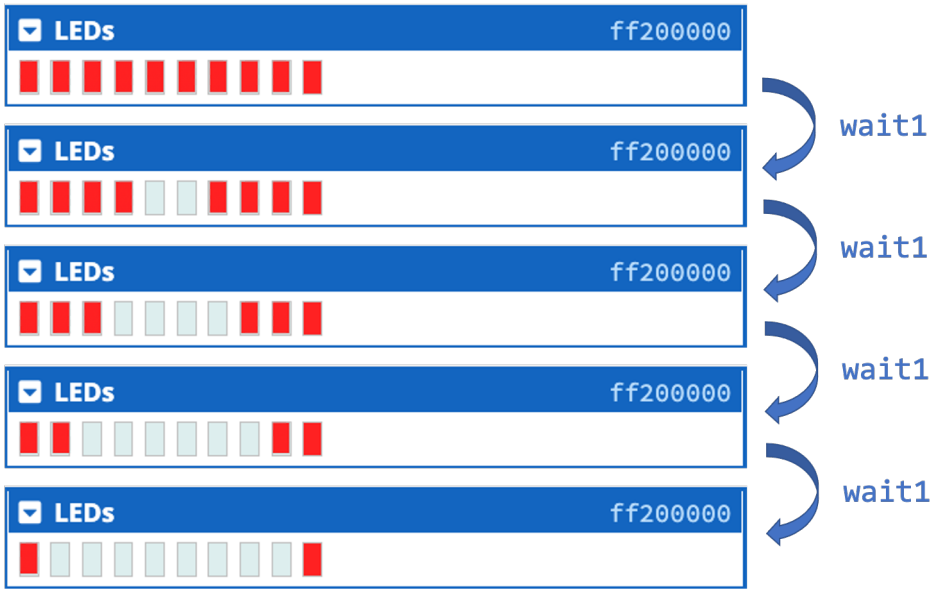
  Valeur recommandée : 0x100000 (vous pouvez essayer d'autres valeurs si ça ne convient pas)

- Ecrire un sous-programme wait qui permet d'attendre un laps de temps multiple de celui de wait1. Ce sous-programme reçoit en paramètre, dans r0, le nombre de fois où il doit appeler wait1.

4.4.2 Fonctions de simulation de la porte de l'ascenseur (LEDs)

La position de la porte de l'ascenseur sera visualisée par les LEDS : toutes les LEDs allumées quand la porte est fermée et les 8 LEDs centrales éteintes quand la porte est ouverte.

- Ecrire un sous-programme ouvrir_porte qui contrôle le passage de la position fermée à la position ouverte selon le schéma ci-dessous. Ce sous-programme n'a pas de paramètre. Utiliser le sous-programme wait1 pour imposer un délai entre deux positions consécutives.



- De manière similaire, écrire un sous-programme fermer_porte qui contrôle le passage de la position ouverte à la position fermée.

4.4.3 Fonctions d'affichage sur les afficheurs 7-segments

L'afficheur 0 (celui qui se trouve le plus à droite) servira à afficher l'étage auquel se trouve la cabine de l'ascenseur.

- Ecrire un sous-programme affiche_étage qui reçoit en paramètre, dans r0, le numéro d'un étage, et l'affiche.

L'afficheur 5 (celui qui se trouve le plus à gauche) servira à visualiser le mouvement de la cabine. Il sera éteint lorsque la cabine est arrêtée à un étage, et affichera deux symboles différents quand elle sera en train de monter ou descendre (par exemple, 'u' pour up et 'd' pour down).

- Ecrire un sous-programme affiche_mouv qui reçoit en paramètre, dans r0, le mouvement (par exemple 0 pour l'arrêt, 1 pour la montée, 2 pour la descente, et), et l'affiche.

4.4.4 Fonctions de lecture des boutons poussoirs

- Ecrire un sous-programme qui lit le registre des boutons poussoirs, pour déterminer sur lesquels on a appuyé, et réinitialise ce registre à 0 en écrivant des 1 dans les 4 bits. On veut que ce sous-programme soit non bloquant : si aucun bouton n'a été pressé depuis la dernière réinitialisation, il renvoie 0.

4.4.5 Programme de simulation de l'ascenseur

- En utilisant les sous-programmes écrits dans les questions précédentes, écrire un programme qui simule le fonctionnement d'un ascenseur qui dessert 4 étages (de 0 à 3). Son fonctionnement est le suivant :
  - au départ, la cabine est arrêtée à l'étage 0 et la porte est fermée
  - à tout moment, le mouvement de la cabine (arrêt, montée, descente) est indiqué sur l'afficheur 7-segments n°5 (le plus à gauche). Si la cabine est arrêtée, l'étage correspondant est indiqué sur l'afficheur 7-segments n°0 (le plus à droite)
  - pour demander le déplacement de la cabine vers un étage donné, on presse (2 clics) le bouton poussoir correspondant
  - l'ascenseur ne change de direction que lorsqu'il n'y a plus d'étage demandé dans la direction courante (et si un étage est demandé dans l'autre direction).